

Fiche de révision — Big Data & BI

Concours ANP — Cadre Data Scientist

Table des matières

1	Hadoop, HDFS, MapReduce	2
1.1	Pourquoi Hadoop existe	2
1.2	HDFS (Hadoop Distributed File System)	2
1.3	MapReduce	2
2	Spark	4
2.1	Pourquoi Spark	4
2.2	RDD et DataFrame	4
2.3	Spark SQL et Spark Streaming	4
2.4	Architecture cluster de Spark (Driver, Cluster Manager, Executors)	5
2.5	Flux Spark complet, du SparkContext au résultat final (théorie)	6
3	Kafka	8
3.1	Rôle de Kafka	8
3.2	Concepts clés	8
4	Architecture bout en bout	9
4.1	Schéma général	9
5	Business Intelligence (concepts)	10
5.1	Cube OLAP, dimensions et mesures	10
5.2	Schéma en étoile vs schéma en flocon	10
5.3	Granularité et opérations OLAP	10
5.4	ETL vs ELT	10
5.5	Data Warehouse vs Data Mart	11
5.6	KPI et tableau de bord	11

1 Hadoop, HDFS, MapReduce

1.1 Pourquoi Hadoop existe

- Constat de départ : au-delà d'un certain volume, les données ne tiennent plus sur un seul disque et un seul serveur ne suffit plus à les traiter dans un temps raisonnable.
- **Hadoop** : framework open-source qui permet de *stocker* et de *traiter* de très gros volumes de données en les répartissant sur un **cluster** (ensemble de machines standard, pas de matériel spécialisé coûteux).
- Idée fondatrice : plutôt que de déplacer les données vers le calcul, on déplace le calcul vers les données (chaque machine traite la portion de données qu'elle stocke déjà localement).
- Deux briques centrales à connaître : **HDFS** (stockage) et **MapReduce** (traitement).

1.2 HDFS (Hadoop Distributed File System)

- **HDFS** : système de fichiers distribué qui découpe chaque fichier en **blocs** (typiquement 128 Mo) et les répartit sur plusieurs machines du cluster.
- **Réplication** : chaque bloc est dupliqué (par défaut 3 copies) sur des machines différentes, pour garantir la disponibilité des données même si une machine tombe en panne.
- Architecture maître/esclave :
 - **NameNode** (maître) : ne stocke pas les données elles-mêmes, mais les *métadonnées* (quel fichier, quels blocs, sur quelles machines).
 - **DataNode** (esclave) : stocke réellement les blocs de données et exécute les instructions du NameNode.
- Conçu pour des fichiers volumineux, un usage *write-once / read-many*, et une tolérance aux pannes matérielles fréquentes sur un grand cluster.

Lecture du schéma (figure 1) :

- Le **Client** ne parle au **Namenode** que pour des opérations sur les métadonnées (*où se trouve tel fichier ?*) — le Namenode ne transmet jamais lui-même la donnée.
- Une fois la localisation connue, le Client échange directement avec les **Datanodes** concernés pour le **Read** (lecture) ou le **Write** (écriture) des blocs.
- La **Replication** (flèche entre Datanodes de racks différents) se fait entre Datanodes, sans repasser par le Client ni par le Namenode : c'est ce qui garantit la disponibilité des blocs même si un Datanode ou un rack entier tombe en panne.

1.3 MapReduce

- **MapReduce** : modèle de programmation pour traiter en parallèle de gros volumes de données stockées sur HDFS, en deux étapes.
- **Map** : chaque machine du cluster traite en parallèle sa portion de données locale et produit des paires (*clé, valeur*) intermédiaires.
- **Shuffle & Sort** (étape intermédiaire) : les paires sont regroupées et triées par clé, puis redistribuées vers les bonnes machines pour l'étape suivante.
- **Reduce** : chaque machine agrège les valeurs reçues pour une même clé (somme, comptage, moyenne...) et produit le résultat final.
- Exemple classique : compter, pour chaque port, le nombre d'escalles de navires dans un immense fichier de logs — *Map* émet (**port**, 1) pour chaque ligne, *Reduce* additionne les 1 par port.

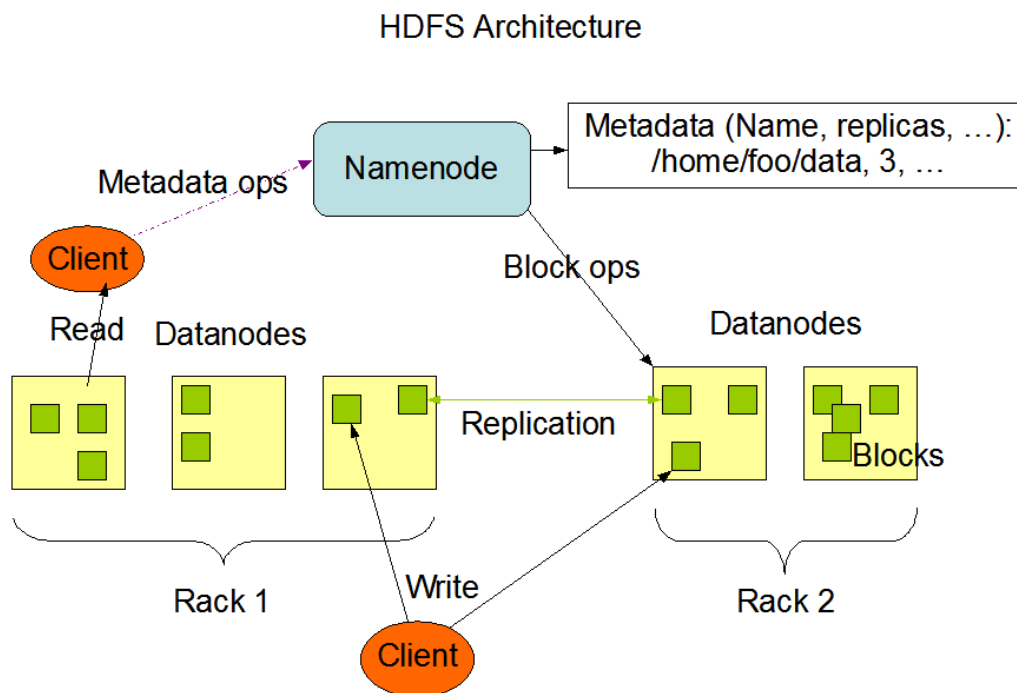


FIGURE 1 – Architecture HDFS : le **Namenode** gère uniquement les métadonnées (*Metadata ops*) tandis que les **Datanodes**, répartis sur plusieurs *Racks*, stockent les blocs réels. Un **Client** contacte le Namenode pour localiser les blocs, puis lit ou écrit directement sur les Datanodes (*Block ops*) ; chaque bloc écrit est répliqué sur un Datanode d'un autre rack pour rester disponible en cas de panne d'un rack entier.

- Limite principale de MapReduce : chaque étape écrit ses résultats intermédiaires sur disque, ce qui le rend lent pour des traitements itératifs ou du temps réel — d'où l'arrivée de Spark (section suivante), qui privilégie le traitement en mémoire.

Lecture du schéma (figure 2) :

- À gauche, les cylindres représentent les blocs de données source (typiquement stockés sur HDFS), traités indépendamment et en parallèle par plusieurs tâches **Map**.
- Chaque tâche Map émet des paires (*clé, valeur*), représentées ici par des couleurs (bleu, vert, jaune) : une couleur = une clé.
- L'étape **Shuffle** (les lignes croisées) redistribue *sur le réseau* toutes les paires de même couleur (même clé) vers une seule et même tâche **Reduce** — c'est l'étape la plus coûteuse du pipeline car elle implique des transferts de données entre machines.
- Chaque tâche Reduce n'agrège que les valeurs de *sa* clé et écrit son résultat, avant que tous les résultats ne soient rassemblés dans le fichier de sortie final.

Question type à savoir répondre à l'oral :

« Quelle est la différence entre HDFS et MapReduce ? » — HDFS répond à la question *où stocker*, MapReduce répond à la question *comment traiter* ; les deux sont complémentaires au sein de Hadoop.

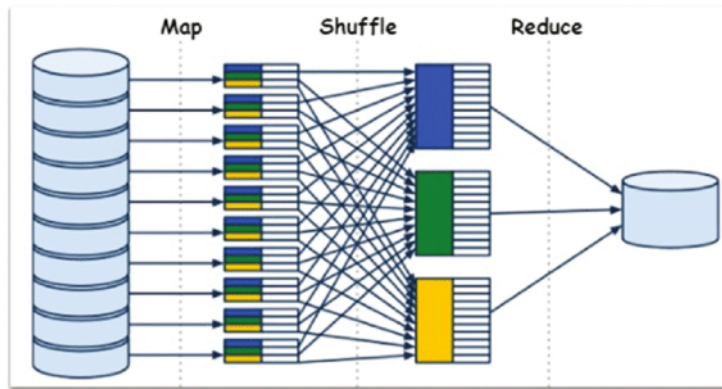


FIGURE 2 – Pipeline MapReduce : chaque partition de données source est traitée en parallèle par une tâche **Map**, qui produit des paires clé/valeur (couleurs); l'étape **Shuffle** redistribue ces paires pour regrouper toutes les valeurs d'une même clé sur le même nœud; chaque tâche **Reduce** agrège ensuite les valeurs reçues pour produire le résultat final consolidé.

2 Spark

2.1 Pourquoi Spark

- **Apache Spark** : moteur de traitement distribué, plus rapide que MapReduce car il privilégie le traitement **en mémoire (RAM)** plutôt que l'écriture disque systématique entre chaque étape.
- Particulièrement adapté aux traitements **itératifs** (ex. algorithmes de Machine Learning qui relisent plusieurs fois les mêmes données) et au **quasi temps réel**.
- Peut lire des données depuis HDFS, mais aussi depuis d'autres sources (bases de données, fichiers cloud, Kafka — voir section suivante) : Spark remplace la couche de *traitement*, pas nécessairement la couche de *stockage*.

2.2 RDD et DataFrame

- **RDD (Resilient Distributed Dataset)** : structure de données de base de Spark — une collection d'objets répartie sur le cluster, tolérante aux pannes (*resilient*) et manipulée via des opérations distribuées (map, filter, reduce...).
- Limite du RDD : pas de structure en colonnes nommées, donc pas d'optimisation automatique des requêtes.
- **DataFrame** : structure de données tabulaire (lignes/colonnes nommées, proche d'une table SQL ou d'un DataFrame pandas), construite au-dessus des RDD, avec un moteur d'optimisation des requêtes (Catalyst) — généralement préféré aux RDD bruts en pratique pour sa simplicité et ses performances.

2.3 Spark SQL et Spark Streaming

- **Spark SQL** : module permettant d'interroger des DataFrames avec des requêtes SQL classiques, en plus de l'API programmatique.
- **Spark Streaming** : module de traitement de flux de données en continu (quasi temps réel), en découpant le flux en micro-batches traités successivement — utile par exemple pour traiter en continu des événements d'arrivée de navires ou de capteurs portuaires.

2.4 Architecture cluster de Spark (Driver, Cluster Manager, Executors)

Pour comprendre *comment* Spark exécute réellement un traitement (batch ou streaming) sur un cluster, il faut connaître quatre rôles :

- **Driver Program** : le processus qui exécute le code principal de l'application Spark (celui écrit par le développeur). C'est lui qui découpe le traitement en tâches (*tasks*) et coordonne tout le reste — il ne traite pas lui-même les données en masse, il orchestre.
- **SparkContext** : point d'entrée créé par le Driver, qui représente la connexion de l'application au cluster et permet de demander des ressources.
- **Cluster Manager** : composant qui alloue les ressources (CPU, mémoire) du cluster aux applications Spark qui en font la demande. Peut être Spark autonome, YARN (l'ordonnanceur historique de Hadoop) ou Kubernetes selon l'environnement.
- **Worker Nodes** : les machines du cluster qui exécutent réellement le travail, chacune hébergeant un ou plusieurs **Task Manager** (exécuteurs) qui font tourner les tâches en parallèle et gardent les données en mémoire entre les étapes.

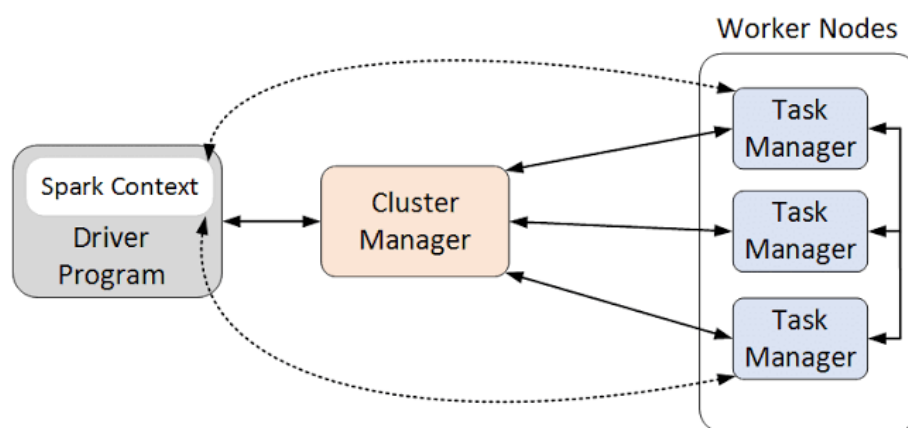


FIGURE 3 – Architecture cluster de Spark (Streaming). Le **Driver Program** (contenant le **Spark Context**) négocie les ressources auprès du **Cluster Manager**, qui répartit le travail sur les **Worker Nodes** : chaque nœud exécute un **Task Manager** chargé d'exécuter les tâches en parallèle.

Déroulé complet d'une exécution (figure 3) :

1. Le **Driver Program** démarre et crée un **Spark Context** ; il analyse le code de l'application et construit un plan d'exécution découpé en étapes et en tâches.
2. Le Driver négocie auprès du **Cluster Manager** les ressources nécessaires (combien de CPU/mémoire, sur combien de machines).
3. Le Cluster Manager alloue des **Task Managers** sur les **Worker Nodes** disponibles — ce sont eux qui exécuteront réellement le travail.
4. Le Driver envoie les tâches à exécuter directement aux Task Managers (flèches Driver ↔ Worker Nodes) ; chaque Task Manager traite sa portion de données en parallèle, en mémoire autant que possible.
5. Pour **Spark Streaming** spécifiquement : le flux entrant est découpé en *micro-batches* (petites fenêtres de temps, ex. toutes les 2 secondes) ; chaque micro-batch est traité comme un mini-job Spark classique en suivant exactement ce même schéma Driver / Cluster Manager / Worker Nodes, ce qui permet à Spark de traiter un flux continu sans architecture séparée pour le temps réel.

Point important pour l'oral : le Driver reste actif et coordonne *pendant toute la durée* de l'application (pas seulement au lancement) — c'est pourquoi, en cas de panne du Driver, toute l'application Spark est perdue, alors qu'une panne d'un seul Worker Node peut être compensée en réattribuant ses tâches à un autre nœud.

2.5 Flux Spark complet, du SparkContext au résultat final (théorie)

Vue d'ensemble, **sans code**, du cycle de vie complet d'une application Spark — du démarrage jusqu'au résultat exploitable, en reliant chaque étape à l'architecture cluster vue plus haut (Driver / Cluster Manager / Worker Nodes) et au schéma bout en bout Sources → Kafka → Spark → Data Lake → Data Warehouse (section 4).

1. **Création du SparkContext (point de départ).** L'application démarre et crée un **SparkContext** (ou **SparkSession**, son équivalent moderne pour les DataFrames) — c'est l'objet qui représente la connexion de l'application au cluster. Sans lui, aucune opération distribuée n'est possible.
2. **Négociation des ressources.** Le **Driver Program** (qui héberge le SparkContext) contacte le **Cluster Manager** pour demander des ressources (CPU, mémoire). Le Cluster Manager alloue alors des **Task Managers** (exécuteurs) sur les **Worker Nodes** disponibles.
3. **Définition de la source (lecture).** L'application indique *d'où* lire les données — par exemple un topic Kafka pour un flux d'événements d'arrivées de navires, ou des fichiers sur HDFS/Data Lake pour un traitement batch. À ce stade, **aucune donnée n'est encore lue** : Spark se contente de noter la source.
4. **Définition des transformations (plan d'exécution paresseux).** Nettoyage, filtrage, agrégation... chaque transformation demandée s'ajoute à un **plan d'exécution**, sans déclencher le moindre calcul immédiat. On parle d'*évaluation paresseuse* (*lazy evaluation*) : Spark attend de connaître *toutes* les étapes avant d'agir, ce qui lui permet d'optimiser globalement le plan (par exemple regrouper un filtre et une agrégation en une seule passe) plutôt que d'exécuter chaque instruction séparément.
5. **Déclenchement réel du traitement (action).** Le calcul ne démarre qu'au moment où une **action** est demandée — typiquement l'écriture du résultat final. C'est à cet instant précis que le Driver traduit le plan d'exécution en tâches concrètes et les envoie aux **Task Managers** des Worker Nodes, qui les exécutent en parallèle et en mémoire.
6. **Découpage en micro-batches (cas du streaming).** Pour un flux continu (Spark Streaming), Spark ne traite jamais « tout le flux d'un coup » : il le découpe en petites fenêtres de temps successives, les **micro-batches** (ex. toutes les quelques secondes). Chaque micro-batch est traité comme un mini-job Spark indépendant, en suivant exactement le même cycle Driver / Cluster Manager / Worker Nodes que pour un job batch classique — c'est ce qui permet à Spark de traiter du temps réel sans moteur séparé.
7. **Écriture du résultat final (sink).** Le résultat de chaque micro-batch (ou du job batch complet) est écrit vers sa destination finale — un Data Lake, un Data Warehouse, ou tout autre système en aval qui alimentera ensuite le ML ou la BI.
8. **Suivi de la progression (tolérance aux pannes).** Spark conserve régulièrement un *point de reprise* (*checkpoint*) indiquant jusqu'où le flux a déjà été traité. En cas de panne, l'application peut redémarrer exactement là où elle s'était arrêtée, sans perdre ni dupliquer de données.
9. **Maintien de la connexion (fin de vie).** Le Driver reste actif et coordonne *pendant toute la durée* de l'application : pour un job batch, il se termine une fois le résultat écrit ; pour un flux streaming, il continue de tourner en boucle jusqu'à un arrêt explicite. Une panne du Driver interrompt toute l'application, alors qu'une panne d'un seul Worker Node peut être compensée en réattribuant ses tâches à un autre nœud disponible.

Question type à savoir répondre à l'oral :

« Pourquoi Spark est-il plus rapide que MapReduce ? » — traitement en mémoire plutôt que ré-écriture disque à chaque étape, ce qui est particulièrement décisif pour les traitements itératifs (ML) et les besoins quasi temps réel.

« Quel est le rôle du Cluster Manager dans Spark ? » — il alloue les ressources du cluster (CPU, mémoire) aux Worker Nodes pour le compte du Driver Program, sans participer lui-même au traitement des données.

« Que se passe-t-il concrètement entre la lecture d'un flux Kafka et l'écriture du résultat ? » — Spark construit d'abord un plan d'exécution paresseux (lecture, transformations, agrégation), qui n'est réellement exécuté qu'au déclenchement de l'action finale (écriture), micro-batch par micro-batch, avec un checkpoint qui garantit la reprise sans perte ni doublon en cas de panne.

3 Kafka

3.1 Rôle de Kafka

- **Apache Kafka** : plateforme de messagerie distribuée conçue pour ingérer, transporter et faire transiter en continu de gros flux de données entre systèmes (bus d'événements en temps réel), avec un fort débit et une faible latence.
- Découple les systèmes *producteurs* de données des systèmes *consommateurs* : ils n'ont pas besoin de se connaître ni d'être synchronisés.

3.2 Concepts clés

- **Producer (producteur)** : application qui envoie (publie) des messages vers Kafka (ex. un système qui émet un événement à chaque arrivée de navire).
- **Consumer (consommateur)** : application qui lit (s'abonne à) les messages depuis Kafka (ex. un job Spark qui traite ces événements en continu).
- **Topic** : catégorie/canal nommé dans lequel les messages sont publiés et organisés (ex. un topic `arrivees_navires`).
- **Partition** : chaque topic est découpé en partitions, réparties sur plusieurs machines, ce qui permet le parallélisme et la scalabilité horizontale.
- **Offset** : identifiant séquentiel (position) d'un message au sein d'une partition, qui permet à un consumer de savoir où il en est dans la lecture et de reprendre exactement où il s'était arrêté.
- **Consumer Group** : ensemble de consumers qui se répartissent la lecture des partitions d'un topic, pour paralléliser la consommation sans traiter deux fois le même message.
- **Replication (réplication)** : chaque partition est dupliquée sur plusieurs machines (comme pour HDFS) afin de garantir la disponibilité des données en cas de panne d'un serveur.

Question type à savoir répondre à l'oral :

« À quoi sert Kafka dans une architecture Big Data ? » — à absorber et transporter en continu des flux d'événements à fort débit entre les sources et les systèmes de traitement (Spark), sans que producteurs et consommateurs soient couplés directement.

4 Architecture bout en bout

4.1 Schéma général

Sources → Kafka → Spark → Data Lake → Data Warehouse → ML / BI

- **Sources** : systèmes qui génèrent la donnée brute (capteurs portuaires, systèmes d'exploitation des terminaux, logs applicatifs, flux AIS des navires, systèmes tiers...).
- **Kafka** : absorbe ces flux en continu et les rend disponibles de façon fiable et découplée pour les systèmes en aval.
- **Spark** : consomme les flux (Spark Streaming) ou des lots de données (batch) depuis Kafka ou le Data Lake, pour nettoyer, transformer et enrichir la donnée (ETL/ELT).
- **Data Lake** : stockage brut, peu coûteux, de très gros volumes de données non forcément structurées, dans leur format d'origine (schema-on-read).
- **Data Warehouse** : stockage de données structurées, nettoyées et modélisées (schema-on-write), organisées pour l'analyse (voir section BI).
- **ML / BI** : couche finale de valorisation — modèles de Machine Learning (ex. prédiction de temps d'attente des navires) et tableaux de bord décisionnels (BI) construits sur le Data Warehouse.

Question type à savoir répondre à l'oral :

« Comment relieriez-vous Hadoop/Spark/Kafka dans une architecture Big Data pour l'ANP ? » — Kafka ingère les flux d'événements portuaires en continu, Spark les traite (streaming ou batch), le résultat est stocké dans un Data Lake puis modélisé dans un Data Warehouse, avant d'alimenter les modèles ML et les dashboards BI.

5 Business Intelligence (concepts)

5.1 Cube OLAP, dimensions et mesures

- **OLAP (On-Line Analytical Processing)** : approche d'analyse de données multidimensionnelle, pensée pour l'exploration rapide et interactive de gros volumes agrégés (par opposition à OLTP, orienté transactions unitaires).
- **Cube OLAP** : représentation des données selon plusieurs **dimensions** croisées (ex. temps × port × type de marchandise), permettant d'analyser une **mesure** sous tous les angles.
- **Dimensions** : axes d'analyse qualitatifs qui permettent de filtrer ou regrouper les données (ex. port, date, type de navire, client).
- **Mesures** : valeurs numériques quantitatives que l'on analyse (ex. volume de conteneurs, chiffre d'affaires, temps d'attente moyen).

5.2 Schéma en étoile vs schéma en flocon

- **Table de faits (fact table)** : table centrale contenant les mesures (les événements mesurables, ex. une escale de navire) ainsi que des clés étrangères vers les tables de dimension.
- **Table de dimension** : table décrivant le contexte d'un axe d'analyse (ex. la table **Port** avec nom, région, statut).
- **Schéma en étoile (star schema)** : une table de faits centrale reliée directement à des tables de dimension *non normalisées* (dénormalisées) — structure simple, requêtes rapides, redondance acceptée.
- **Schéma en flocon (snowflake schema)** : variante où les tables de dimension sont elles-mêmes normalisées en sous-tables — moins de redondance, mais requêtes plus complexes (plus de jointures).

5.3 Granularité et opérations OLAP

- **Granularité** : niveau de détail des données stockées dans la table de faits (ex. par escale individuelle vs agrégé par jour vs agrégé par mois) — plus la granularité est fine, plus l'analyse est flexible mais plus le volume est important.
- **Drill-down** : descendre à un niveau de détail plus fin (ex. du total annuel vers le détail mensuel).
- **Roll-up** : remonter à un niveau plus agrégé (ex. du détail par port vers le total national).
- **Slice** : fixer une dimension à une seule valeur pour analyser une "tranche" du cube (ex. ne garder que l'année 2025).
- **Dice** : filtrer sur plusieurs dimensions simultanément pour obtenir un sous-cube plus restreint.
- **Pivot (rotation)** : changer l'orientation du cube pour croiser les dimensions différemment (ex. permuter lignes et colonnes d'un tableau croisé).

5.4 ETL vs ELT

- **ETL (Extract, Transform, Load)** : les données sont extraites, *transformées* (nettoyage, agrégation, mise en forme) *avant* d'être chargées dans le système cible — approche classique pour un Data Warehouse structuré.
- **ELT (Extract, Load, Transform)** : les données sont chargées brutes d'abord, puis transformées *dans* le système cible — approche plus adaptée aux environnements Big Data/Cloud où la puissance de calcul cible est importante (ex. transformation directement dans le Data Lake ou un entrepôt cloud).

5.5 Data Warehouse vs Data Mart

- **Data Warehouse** : entrepôt de données centralisé, structuré, couvrant l'ensemble de l'organisation, alimenté par de multiples sources.
- **Data Mart** : sous-ensemble du Data Warehouse, restreint à un métier ou un département spécifique (ex. un data mart dédié à la Direction Stratégie et Développement), plus simple et plus rapide à interroger pour un besoin ciblé.

5.6 KPI et tableau de bord

- **KPI (Key Performance Indicator)** : indicateur clé de performance, choisi pour refléter directement un objectif stratégique (ex. taux d'occupation des quais, temps d'attente moyen des navires, volume de trafic par type de marchandise).
- **Tableau de bord (dashboard)** : ensemble de KPI et de visualisations organisés pour permettre un suivi rapide et une aide à la décision, généralement construit sur un Data Warehouse ou Data Mart plutôt que sur la donnée brute.

Question type à savoir répondre à l'oral :

« Quelle est la différence entre un schéma en étoile et un schéma en flocon ? » — le schéma en étoile dénormalise les dimensions pour des requêtes plus simples et rapides, le schéma en flocon les normalise pour réduire la redondance au prix de jointures supplémentaires.